

Whack-a-Mole Game Features and Implementation Logic

1. BP_Target_Mole (Mole)

1.1 Overview and State Machine Design

Purpose

Implements the whack-a-mole game's core object — the mole. Moles periodically pop out of the panel holes, stay, and retract on timeout; when hit by the player's hammer they play a sound and effect and score. Moles are not dynamically spawned and destroyed — 8 are manually placed in the level and repeatedly reused through a state machine (the same idea as the bowling pins' placement).

State machine design

- Hidden: the mole is below the hole with collision off, waiting for the GameManager to activate it; the only state PopUp accepts
- Rising: the Timeline plays forward; can be hit
- Up: fully popped and staying; can be hit, and automatically retracts after the stay timeout
- Hit: struck by the hammer; the state locks and it immediately retracts
- Retreating: the Timeline plays in reverse; returns to Hidden when the retraction completes

Blueprint screenshots

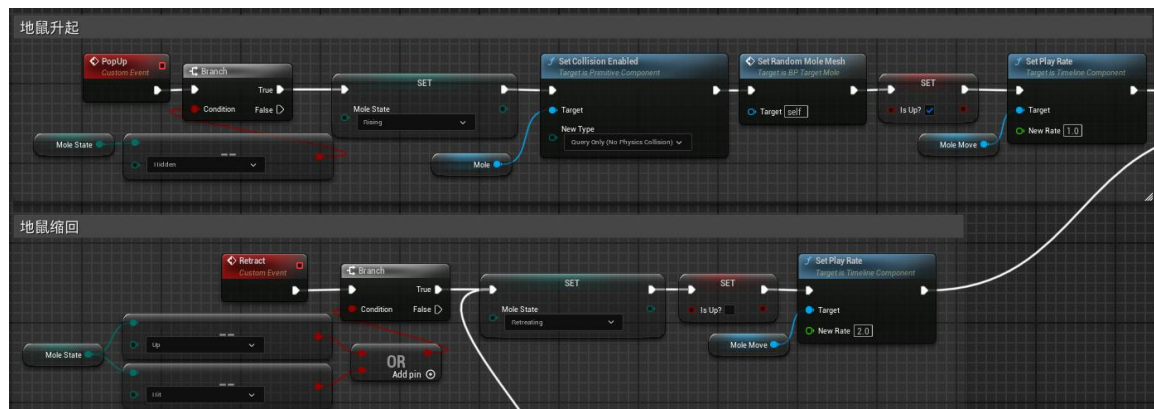


1.2 PopUp and Retract Events

Purpose

The two entry events for the mole's rise and retraction. Both entry points check the state, preventing repeated activation or retraction; they share one Timeline, distinguishing the rise's normal speed from the retraction's fast speed by setting different play rates.

Blueprint screenshots



Key points

- The state checks prevent repeated activation:
 - PopUp only accepts Hidden, never re-activating the same mole
 - Retract accepts Up and Hit and switches straight to Retreating on success, so no repeated retraction occurs
- The Is Up? boolean serves two roles at once: recording the current orientation state + letting the Finished branch decide the animation direction
- Collision turns on at PopUp and off when the retraction completes, preventing the hammer from falsely triggering moles below the holes

1.3 Timeline Rise/Fall Animation Control (MoleMove)

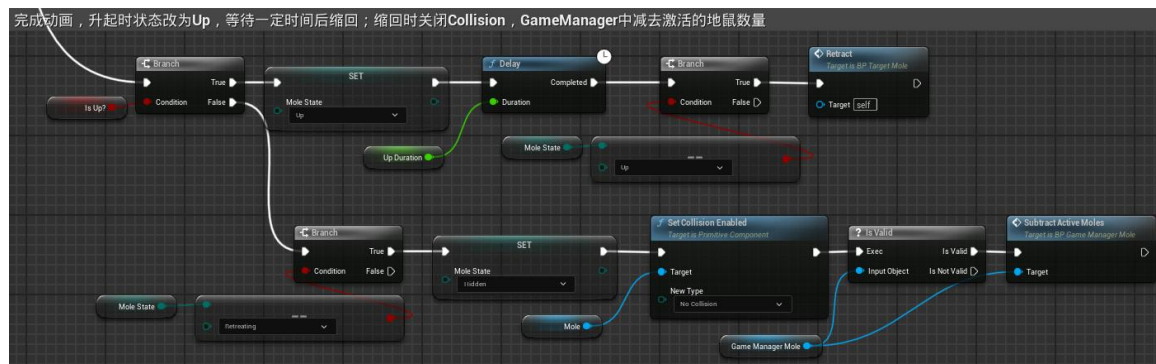
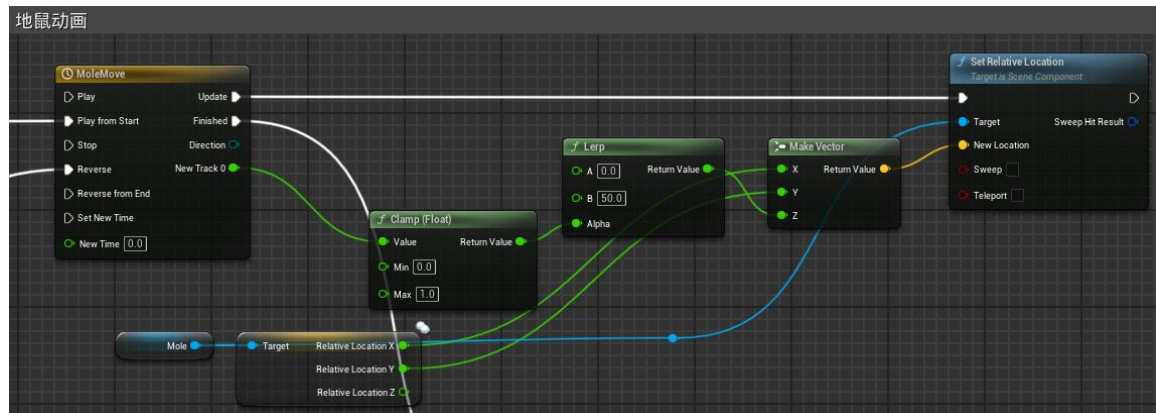
Purpose

Drives the mole's rise and fall with one bidirectionally reused Timeline. Rising runs at normal speed and retracting at 2× rate — retracting faster than popping out matches whack-a-mole's snappy feel.

Play rate setup

The Timeline node itself has no Play Rate pin; drag MoleMove from the Timelines category of the My Blueprint panel into the event graph to get the Timeline component reference, then call its Set Play Rate function. Set the rate to 1.0 (normal) before rising and 2.0 (double) before retracting. The rate is set before calling Play/Reverse, halving the Timeline's total duration when played in reverse, achieving a genuinely fast retraction.

Blueprint screenshots



Notes

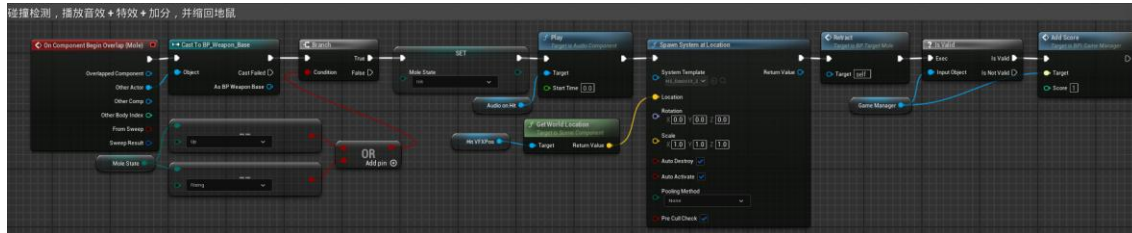
- Reverse from End plays backward from the end; the Finished pin also fires when playback completes (back at 0 seconds)
- The Direction output pin is 1.0 when playing forward and -1.0 in reverse, usable instead of Is Up? to determine the animation direction

1.4 Hit Detection (Hammer Overlap)

Purpose

Detects the hammer hitting the mole, plays the sound and effect, adds the score, and retracts the mole.

Blueprint screenshots



Key points

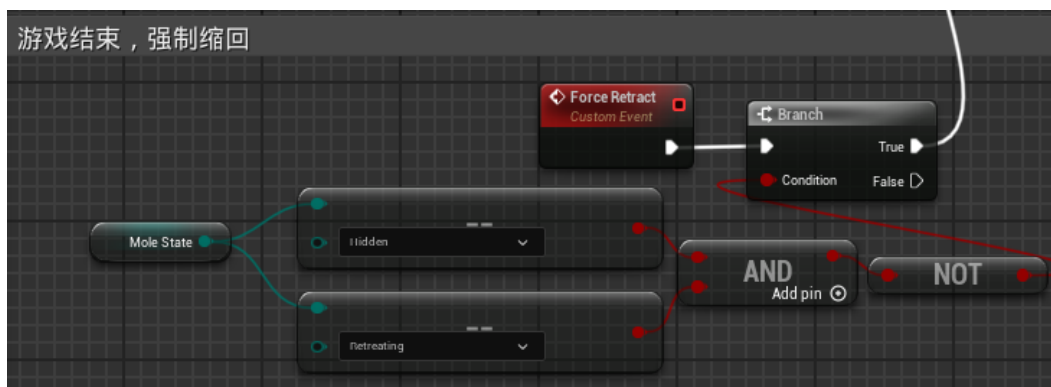
- While grabbed, the hammer's physics is off (kinematically following the controller), and the mole also moves kinematically; two non-physics-simulated objects produce no Hit event, but Overlap events still fire normally (as long as Generate Overlap Events is checked)

1.5 Force Retract (Forced Retraction on Game End)

Purpose

Forces all moles to retract when the game ends. The normal Retract event only handles Up and Hit and does not cover Rising — a mole mid-rise cannot be retracted through the normal flow, so the Force Retract event was written specifically to close that gap.

Blueprint screenshots



Key points

- The difference from normal Retract is the state check: the Force version ignores Up/Hit and additionally allows the Rising state

- On game end, mid-rising moles are forcibly retracted, avoiding moles stuck half-out in the end screen

2. BP_Weapon_Hammer (Hammer)

2.1 Hammer Overrides and Return-on-Release

Purpose

The hammer is the simplest weapon: no projectile logic, Shoot and On Hit Target are overridden to suppress the parent's launch behavior; on release it simply teleports back to its initial position — no destroy-and-respawn, and no need to decide whether to spawn a new weapon.

Blueprint screenshots



Key points

- Teleport checked, Sweep unchecked: the teleport does not check path collision, avoiding the hammer getting stuck on the panel when returning
- Hitting moles relies on the hammer head collider's Overlap event (detected on the mole side); the hammer itself needs no hit logic
- The return design lets the hammer be repeatedly picked up and put down, with zero spawn/destroy cost for the whole game

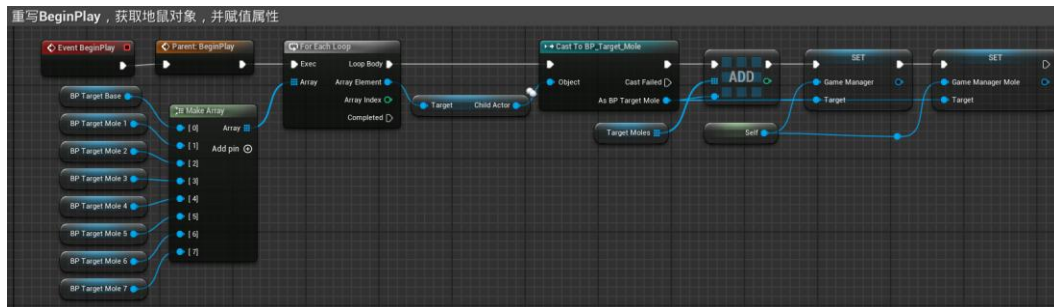
3. GameManager (Whack-a-Mole Game Manager)

3.1 Mole Initialization (BeginPlay)

Purpose

At game start, gets references to the 8 manually placed moles in the level, stores them in an array for the spawn logic, and assigns the GameManager reference to each mole. Exactly the same initialization pattern as the bowling pins.

Blueprint screenshots

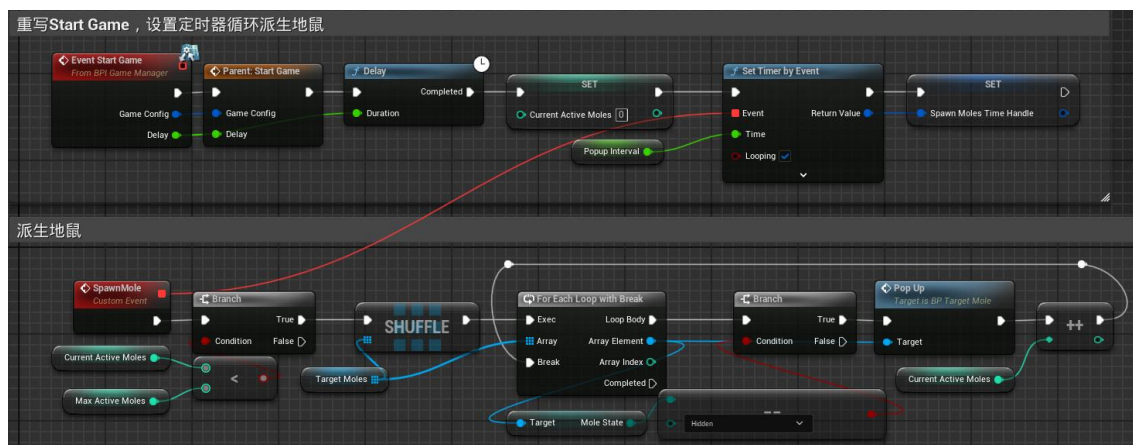


3.2 Timer and Mole Spawning (Start Game / Spawn Mole)

Purpose

After the game starts, a looping timer spawns moles at intervals. Each spawn first randomly shuffles the mole array, then finds the first Hidden mole and activates it, while an active count caps the number of moles on the field.

Blueprint screenshots



Key points

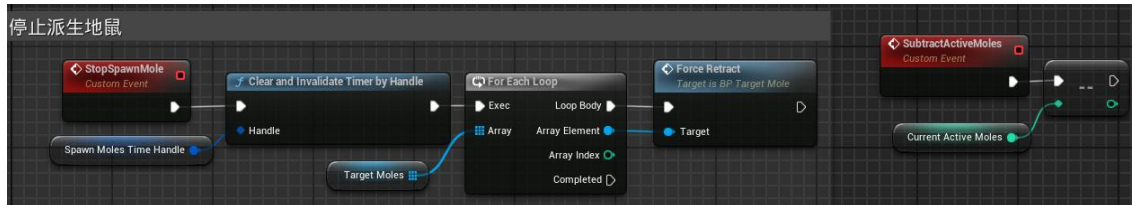
- SHUFFLE + Break achieves random selection: after each shuffle the first available one is taken, so the player cannot predict which will pop up
- The Max Active Moles cap prevents field chaos; at the cap the timer keeps running but skips spawning — simple and reliable
- For Each Loop with Break ends early once the target is found, without iterating the remaining elements

3.3 Stop Spawning and Active Counting (Stop Spawn Mole / Subtract Active Moles)

Purpose

After the countdown ends, mole spawning stops and all moles are forcibly retracted; each mole notifies the GameManager when its retraction completes to decrement the active count, keeping the count in sync with reality.

Blueprint screenshots



Key points

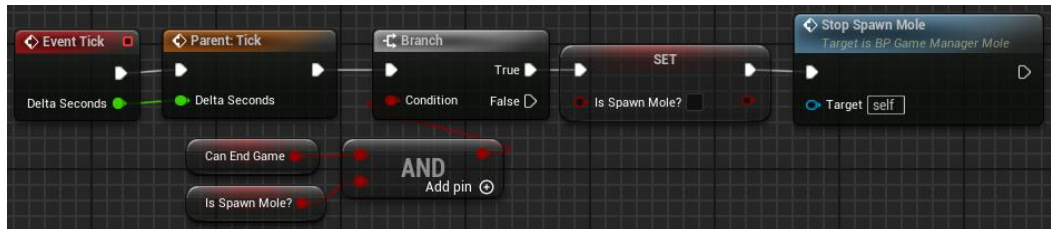
- Counting loop: +1 on spawn, -1 on retraction, so Current Active Moles always equals the active moles on the field

3.4 Tick End Check

Purpose

Whack-a-mole only uses the countdown scoring mode (mole pop-ups are not a fixed number, so attempt-based scoring does not fit). Tick polls whether the countdown has ended and stops spawning when it does. The Is Spawn Mole? variable ensures the stop runs only once, not re-triggered on later frames.

Blueprint screenshots



Key points

- Is Spawn Mole? is a one-shot guard: once false, later frames' AND condition necessarily fails, so Stop Spawn Mole is not repeatedly called every frame
- When the countdown ends, spawning stops immediately (no delay), but the final game end still waits until the player releases the weapon — consistent with the delayed-ending mechanism of the gun and darts waiting for throws to finish

- Is Spawn Mole? is set true at game start and false when spawning stops; the two are maintained in pairs

4. Complete Game Flow

Execution flow

1. The player enters the trigger area, clicks the Start button, and the GameManager runs Start Game
2. After the opening Delay, the active count is zeroed, the looping timer is created, and Is Spawn Mole? is set true
3. The player picks up the hammer (the hammer head's Overlap gains detection capability)
4. Every PopUp Interval seconds the timer triggers SpawnMole, randomly activating one Hidden mole
5. The mole Rises (rate 1.0); after rising it stays Up for Up Duration
6. The player swings and hits an Up/Rising mole → Hit locks the state → sound + effect + score → fast retraction (rate 2.0)
7. If not hit in time → Retract automatically retracts, falling just as fast
8. When the retraction completes → Hidden with collision off → notify the GameManager to decrement the active count
9. The countdown hits zero, Tick detects Can End Game → Is Spawn Mole? set false → Stop Spawn Mole
10. The timer is cleared, all moles are Force Retracted, and spawning fully stops
11. The player releases the hammer, which teleports back to its initial position and runs Check Can End Game; the game finally ends